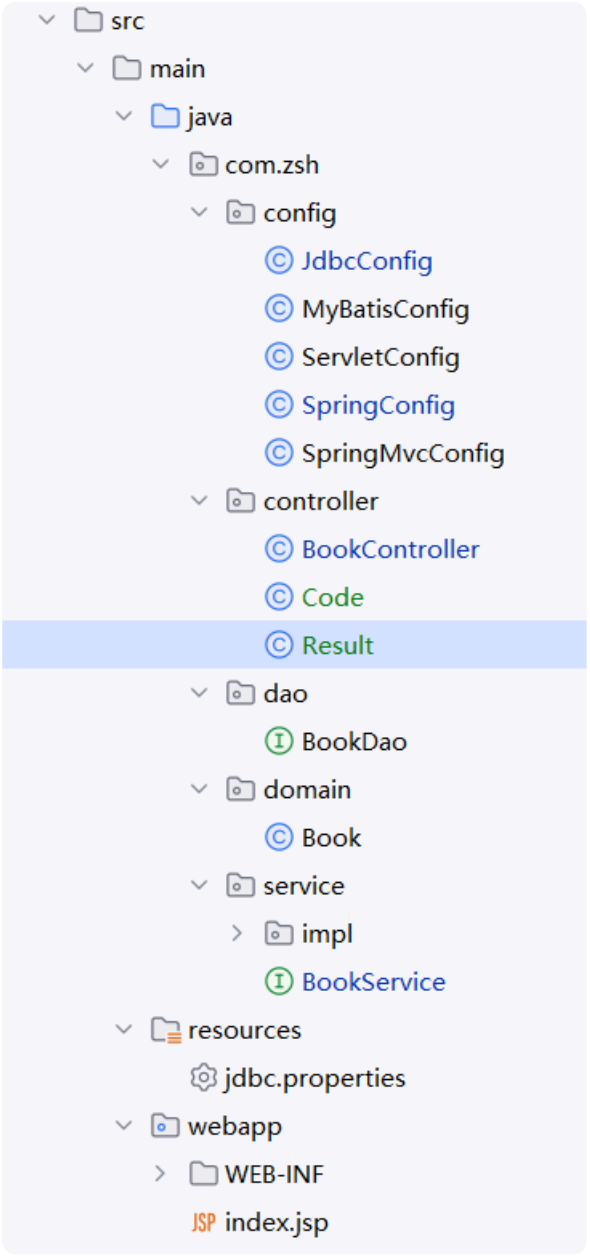
SSM整合

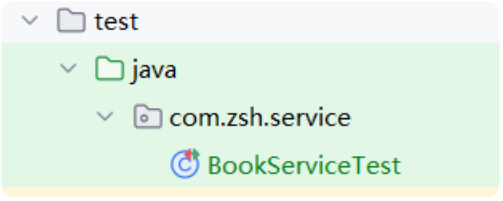
> 鸣谢：黑马程序员
>
> 
>
> 黑马程序员SSM框架教程_Spring+SpringMVC+Maven高级+SpringBo...
> UP 黑马程序员 · 2022-4-20

项目结构：

```
src
├─main
│  ├─java
│  │  └─com
│  │      └─zsh
│  │          ├─config
│  │          │      JdbcConfig.java
│  │          │      MyBatisConfig.java
│  │          │      ServletConfig.java
│  │          │      SpringConfig.java
│  │          │      SpringMvcConfig.java
│  │          │
│  │          ├─controller
│  │          │      BookController.java
│  │          │      Code.java
│  │          │      Result.java
│  │          │
│  │          ├─dao
│  │          │      BookDao.java
│  │          │
```

```
|   |           ├─domain
|   |           |      Book.java
|   |           |
|   |           └─service
|   |               |  BookService.java
|   |               |
|   |               └─impl
|   |                       BookServiceImpl.java
|   |
|   ├─resources
|   |      jdbc.properties
|   |
|   └─webapp
|       |  index.jsp
|       |
|       └─WEB-INF
|              web.xml
|
└─test
    └─java
        └─com
            └─zsh
                └─service
                        BookServiceTest.java
```


src
main
java
com.zsh
config
JdbcConfig
MyBatisConfig
ServletConfig
SpringConfig
SpringMvcConfig
controller
BookController
Code
Result
dao
BookDao
domain
Book
service
impl
BookService
resources
jdbc.properties
webapp
WEB-INF
index.jsp

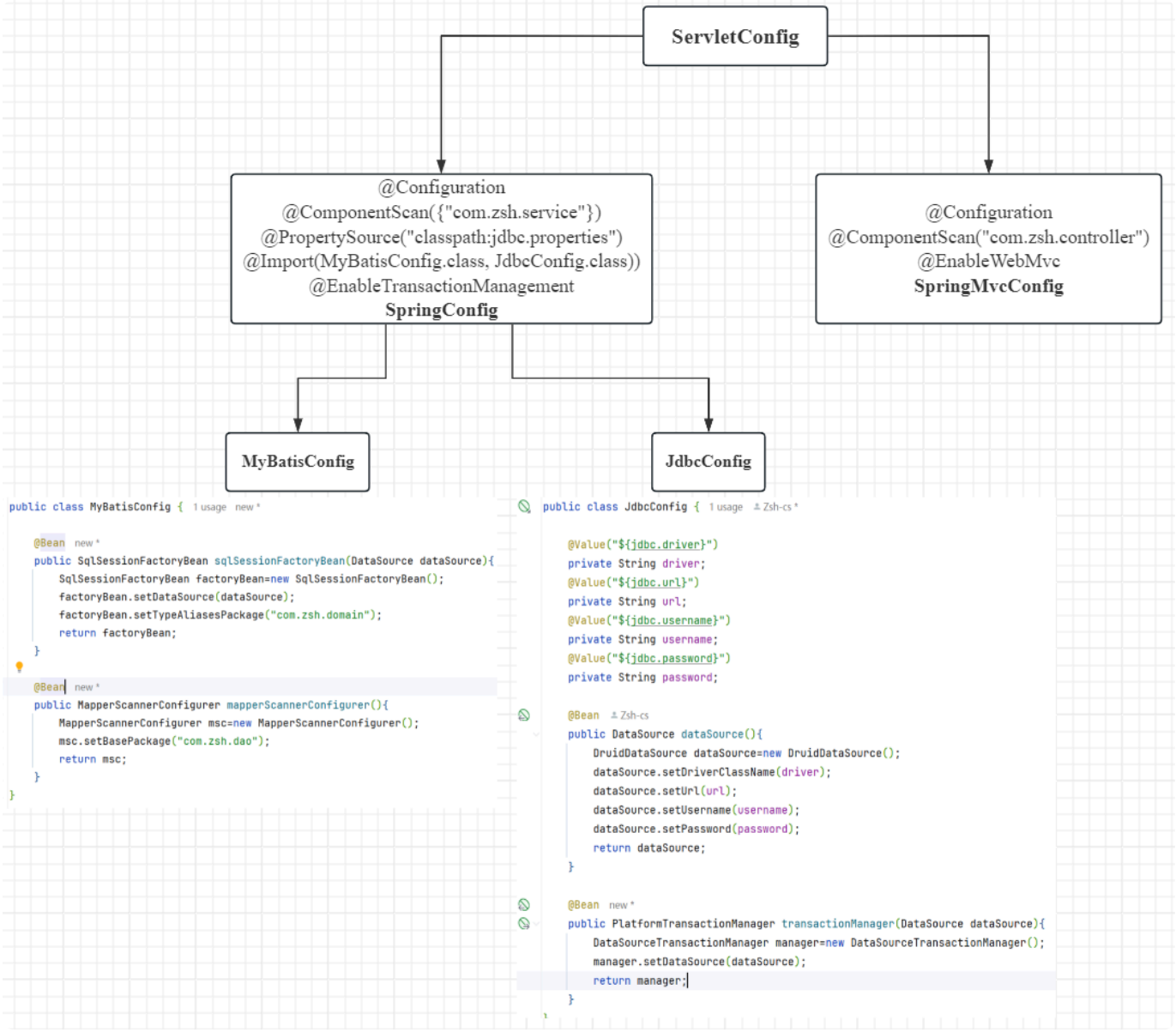

test
java
com.zsh.service
BookServiceTest

Part Ⅰ 后端开发

一、SSM整合流程

1. 创建工程
2. SSM整合配置：
    - Spring：`SpringConfig`
    - SpringMVC：
        - `ServletConfig`
        - `SpringMvcConfig`
    - MyBatis：
        - `MyBatisConfig`
        - `JdbcConfig`
        - `jdbc.properties`
3. 功能模块开发：
    - 表与实体类
    - dao（接口+自动代理）
    - service（接口+实现类）：业务层接口测试（整合`JUnit`）
    - controller：表现层接口测试（`Postman`）

二、`config`包—核心配置类



1. `jdbc.properties` + `JdbcConfig`

- `jdbc.properties`:

```
jdbc.driver=com.mysql.cj.jdbc.Driver
jdbc.url=jdbc:mysql://localhost:3306/ssm_db
jdbc.username=root
jdbc.password=123456
```

- `JdbcConfig`:

```
package com.zsh.config;

```

```java
import com.alibaba.druid.pool.DruidDataSource;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;

import javax.sql.DataSource;

public class JdbcConfig {

    @Value("${jdbc.driver}")
    private String driver;
    @Value("${jdbc.url}")
    private String url;
    @Value("${jdbc.username}")
    private String username;
    @Value("${jdbc.password}")
    private String password;

    @Bean
    public DataSource dataSource(){
        DruidDataSource dataSource=new DruidDataSource();
        dataSource.setDriverClassName(driver);
        dataSource.setUrl(url);
        dataSource.setUsername(username);
        dataSource.setPassword(password);
        return dataSource;
    }

    @Bean
    public PlatformTransactionManager transactionManager(DataSource dataSource){
        DataSourceTransactionManager manager=new DataSourceTransactionManager();
        manager.setDataSource(dataSource);
        return manager;
    }
}
```

2. MyBatisConfig

```java
package com.zsh.config;

import org.mybatis.spring.SqlSessionFactoryBean;
import org.mybatis.spring.mapper.MapperScannerConfigurer;
import org.springframework.context.annotation.Bean;

import javax.sql.DataSource;

public class MyBatisConfig {

    @Bean
    public SqlSessionFactoryBean sqlSessionFactoryBean(DataSource dataSource){
        SqlSessionFactoryBean factoryBean=new SqlSessionFactoryBean();
        factoryBean.setDataSource(dataSource);
        factoryBean.setTypeAliasesPackage("com.zsh.domain");
        return factoryBean;
    }

    @Bean
    public MapperScannerConfigurer mapperScannerConfigurer(){
        MapperScannerConfigurer msc=new MapperScannerConfigurer();
        msc.setBasePackage("com.zsh.dao");
        return msc;
    }
}
```

3. SpringConfig

```java
package com.zsh.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Import;
import org.springframework.context.annotation.PropertySource;

@Configuration
@ComponentScan({"com.zsh.service"})
```

```
@PropertySource("classpath:jdbc.properties")
@Import({MyBatisConfig.class, JdbcConfig.class})
@EnableTransactionManagement
public class SpringConfig {
}
```

4. SpringMvcConfig

```
package com.zsh.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;

@Configuration
@ComponentScan("com.zsh.controller")
@EnableWebMvc
public class SpringMvcConfig {
}
```

5. ServletConfig

```
package com.zsh.config;

import org.springframework.web.filter.CharacterEncodingFilter;
import
org.springframework.web.servlet.support.AbstractAnnotationConfigDispatc
herServletInitializer;

import javax.servlet.Filter;

public class ServletConfig extends
AbstractAnnotationConfigDispatcherServletInitializer {
    @Override
    protected Class<?>[] getRootConfigClasses() {
        return new Class[]{SpringConfig.class};
    }
```

```
13
14      @Override
15      protected Class<?>[] getServletConfigClasses() {
16          return new Class[]{SpringMvcConfig.class};
17      }
18
19      @Override
20      protected String[] getServletMappings() {
21          return new String[]{"/"};
22      }
23
24      // POST请求中文乱码处理：设置过滤器
25      @Override
26      protected Filter[] getServletFilters() {
27          CharacterEncodingFilter filter=new CharacterEncodingFilter();
28          filter.setEncoding("UTF-8");
29          return new Filter[]{filter};
30      }
31  }
```

三、功能模块开发

1. Book

```
1   package com.zsh.domain;
2
3   public class Book {
4       private Integer id;
5       private String type;
6       private String name;
7       private String description;
8
9       public Book(){}
10
11      public Book(Integer id, String type, String name, String
    description) {
12          this.id = id;
13          this.type = type;
```

```
        this.name = name;
        this.description = description;
    }

    @Override
    public String toString() {
        return "Book{" +
                "id=" + id +
                ", type='" + type + '\'' +
                ", name='" + name + '\'' +
                ", description='" + description + '\'' +
                '}';
    }

    // 省略getter&setter

}
```

2. BookDao

```
package com.zsh.dao;

import com.zsh.domain.Book;
import org.apache.ibatis.annotations.*;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface BookDao {

    @Insert("insert into books (type, name, description) value (#{type},#{name},#{description})")
    int save(Book book);

    @Delete("delete from books where id=#{id}")
    int delete(Integer id);

    @Update("update books set type=#{type},name=#{name},description=#{description} where id=#{id}")
    int update(Book book);
```

```
20
21      @Select("select * from books where id=#{id}")
22      Book getById(Integer id);
23
24      @Select("select * from books")
25      List<Book> getAll();
26  }
27
```

3. `BookService` + `BookServiceImpl` + `BookServiceTest`

- `BookService`:

```
1  package com.zsh.service;
2
3  import com.zsh.domain.Book;
4  import org.springframework.transaction.annotation.Transactional;
5
6  import java.util.List;
7
8  @Transactional
9  public interface BookService {
10     boolean save(Book book);
11     boolean delete(Integer id);
12     boolean update(Book book);
13     Book getById(Integer id);
14     List<Book> getAll();
15 }
```

- `BookServiceImpl`:

```
1  package com.zsh.service.impl;
2
3  import com.zsh.controller.Code;
4  import com.zsh.dao.BookDao;
5  import com.zsh.domain.Book;
6  import com.zsh.exception.BuisnessException;
7  import com.zsh.exception.SystemException;
8  import com.zsh.service.BookService;
9  import org.springframework.beans.factory.annotation.Autowired;
10 import org.springframework.stereotype.Service;
```

```
11
12 import java.util.List;
13
14 @Service
15 public class BookServiceImpl implements BookService {
16
17     @Autowired
18     private BookDao bookDao;
19
20     @Override
21     public boolean save(Book book) {
22         return bookDao.save(book) > 0 ? true : false;
23     }
24
25     @Override
26     public boolean delete(Integer id) {
27         return bookDao.delete(id) > 0 ? true : false;
28     }
29
30     @Override
31     public boolean update(Book book) {
32         return bookDao.update(book) > 0 ? true : false;
33     }
34
35     @Override
36     public Book getById(Integer id) {
37         return bookDao.getById(id);
38     }
39
40     @Override
41     public List<Book> getAll() {
42         return bookDao.getAll();
43     }
44 }
```

- `BookServiceTest`：

```
1 package com.zsh.service;
2
3 import com.zsh.config.SpringConfig;
4 import com.zsh.domain.Book;
5 import org.junit.Test;
6 import org.junit.runner.RunWith;
7 import org.springframework.beans.factory.annotation.Autowired;
```

```java
import org.springframework.test.context.ContextConfiguration;
import
org.springframework.test.context.junit4.SpringJUnit4ClassRunner;

import java.util.List;

@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration(classes = SpringConfig.class)
public class BookServiceTest {

    @Autowired
    private BookService bookService;

    @Test
    public void testGetById(){
        Book book = bookService.getById(1);
        System.out.println(book);
    }

    @Test
    public void testGetAll(){
        List<Book> books=bookService.getAll();
        System.out.println(books);
    }
}
```

4. `BookController`

```java
package com.zsh.controller;

import com.zsh.domain.Book;
import com.zsh.service.BookService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/books")
public class BookController {

```

```
14      @Autowired
15      private BookService bookService;
16
17      @PostMapping
18      public boolean save(@RequestBody Book book) {
19          return bookService.save(book);
20      }
21
22      @DeleteMapping("/{id}")
23      public boolean delete(@PathVariable Integer id) {
24          return bookService.delete(id);
25      }
26
27      @PutMapping
28      public boolean update(@RequestBody Book book) {
29          return bookService.update(book);
30      }
31
32      @GetMapping("/{id}")
33      public Book getById(@PathVariable Integer id) {
34          return bookService.getById(id);
35      }
36
37      @GetMapping
38      public List<Book> getAll() {
39          return bookService.getAll();
40      }
41
42  }
```

PartII 表现层与前端数据交互

一、设置统一的数据返回结果类`Result`

> **Tip**

> `Result`类中的字段并不是固定的，可以根据需要自行增减，同时要提供若干个构造方法，方便操作。

```java
package com.zsh.controller;

public class Result {
    private Integer code;// 状态码
    private Object data;// 数据
    private String msg;// 响应消息

    public Result() {

    }

    public Result(Integer code, Object data) {
        this.code = code;
        this.data = data;
    }

    public Result(Integer code, Object data, String msg) {
        this.code = code;
        this.data = data;
        this.msg = msg;
    }

    // 省略getter&setter
}
```

二、设置统一的数据返回结果状态码类`Code`

```java
package com.zsh.controller;

public class Code {
    // 结尾1代表成功，0代表失败

    public static final Integer SAVE_OK = 20011;
    public static final Integer DELETE_OK = 20021;
    public static final Integer UPDATE_OK = 20031;
    public static final Integer GET_OK = 20041;
```

```
10
11      public static final Integer SAVE_ERR = 20010;
12      public static final Integer DELETE_ERR = 20020;
13      public static final Integer UPDATE_ERR = 20030;
14      public static final Integer GET_ERR = 20040;
15  }
```

三、重构`BookController`

```
1   package com.zsh.controller;
2
3   import com.zsh.domain.Book;
4   import com.zsh.service.BookService;
5   import org.springframework.beans.factory.annotation.Autowired;
6   import org.springframework.web.bind.annotation.*;
7
8   import java.util.List;
9
10  @RestController
11  @RequestMapping("/books")
12  public class BookController {
13
14      @Autowired
15      private BookService bookService;
16
17      @PostMapping
18      public Result save(@RequestBody Book book) {
19          boolean flag = bookService.save(book);
20          return new Result(flag ? Code.SAVE_OK : Code.SAVE_ERR, flag);
21      }
22
23      @DeleteMapping("/{id}")
24      public Result delete(@PathVariable Integer id) {
25          boolean flag = bookService.delete(id);
26          return new Result(flag ? Code.DELETE_OK : Code.DELETE_ERR,
    flag);
27      }
28
29      @PutMapping
30      public Result update(@RequestBody Book book) {
31          boolean flag = bookService.update(book);
```

```
32          return new Result(flag ? Code.UPDATE_OK : Code.UPDATE_ERR,
    flag);
33      }
34
35      @GetMapping("/{id}")
36      public Result getById(@PathVariable Integer id) {
37          Book book = bookService.getById(id);
38          Integer code = book != null ? Code.GET_OK : Code.GET_ERR;
39          String msg = book != null ? "数据查询成功" : "数据查询失败，请重
    试";
40          return new Result(code, book, msg);
41      }
42
43      @GetMapping
44      public Result getAll() {
45          List<Book> books = bookService.getAll();
46          Integer code = books != null ? Code.GET_OK : Code.GET_ERR;
47          String msg = books != null ? "数据查询成功" : "数据查询失败，请重
    试";
48          return new Result(code, books, msg);
49      }
50
51  }
```

PartⅢ 项目异常处理

一、异常处理器

1.异常的常见位置和诱因

常见位置	诱因
框架内部异常	使用不合规
数据层异常	外部服务器故障，如服务器访问超时

常见位置	诱因
业务层异常	业务逻辑书写错误，如遍历业务书写操作导致索引越界异常
表现层异常	数据收集、校验等规则错误，如不匹配的数据类型间导致异常
工具类异常	工具类书写不严谨不够健壮，如长期未释放某个必须释放的连接

2.异常处理思路

各个层级均有可能出现异常，我们将所有异常都抛出到表现层进行统一处理。

表现层处理异常，若每个方法中单独书写，代码臃肿且意义不大，如何解决？——**AOP**思想。

3.异常处理器

```java
package com.zsh.controller;

import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

/**
 * 项目异常统一处理
 */
@RestControllerAdvice// REST风格
public class ProjectExceptionAdvice {

    @ExceptionHandler(Exception.class)// 拦截所有异常并处理
    public Result handleException(Exception e) {
        System.out.println("catch an exception...");
        return new Result(666, null, "catch an exception");
    }
}
```

二、项目异常处理方案

1.项目异常分类

- **业务异常(BusinessException)**：
  - 规范的用户行为操作产生的异常，如用户在年龄输入框输入了“十八”而非“18”。。
  - 不规范的用户行为操作产生的异常，如用户在地址栏输入了`http://localhost/users/heihei`而非`http://localhost/books/1`。
- **系统异常(SystemException)**：项目运行过程中可预计且无法避免的异常，如服务器宕机。
- **其他异常(Exception)**：编程人员未预料到的异常，如找不到指定路径的文件。

2.处理方案

- **业务异常**：发送**对应**消息给用户，提醒用户进行规范操作。
- **系统异常**：
  - 发送**固定**消息给用户，安抚用户。
  - 发送**特定**消息给运维人员，提醒他们维护。
  - 记录日志。
- **其他异常**：
  - 发送**固定**消息给用户，安抚用户。
  - 发送**特定**消息给开发人员，提醒他们将此异常纳入预期范围内。
  - 记录日志。

3.代码实现

3.1 定义`BusinessException`和`SystemException`

- `BusinessException`：

```
1  package com.zsh.exception;
```

```
2
3  public class BuisnessException extends RuntimeException {
4      private Integer code;
5
6      public BuisnessException(Integer code, String message) {
7          super(message);
8          this.code = code;
9      }
10
11     public BuisnessException(Integer code, String message,
   Throwable cause) {
12         super(message, cause);
13         this.code = code;
14     }
15
16     public Integer getCode() {
17         return code;
18     }
19
20     public void setCode(Integer code) {
21         this.code = code;
22     }
23 }
```

- `SystemException`:

```
1  package com.zsh.exception;
2
3  public class SystemException extends RuntimeException {
4      private Integer code;
5
6      public SystemException(Integer code, String message) {
7          super(message);
8          this.code = code;
9      }
10
11     public SystemException(Integer code, String message, Throwable
   cause) {
12         super(message, cause);
13         this.code = code;
14     }
15
16     public Integer getCode() {
17         return code;
```

```
    }

    public void setCode(Integer code) {
        this.code = code;
    }
}
```

3.2 更新`Code`类

```
package com.zsh.controller;

public class Code {
    // 结尾1代表成功，0代表失败

    public static final Integer SAVE_OK = 20011;
    public static final Integer DELETE_OK = 20021;
    public static final Integer UPDATE_OK = 20031;
    public static final Integer GET_OK = 20041;

    public static final Integer SAVE_ERR = 20010;
    public static final Integer DELETE_ERR = 20020;
    public static final Integer UPDATE_ERR = 20030;
    public static final Integer GET_ERR = 20040;

    public static final Integer SYSTEM_ERR = 50001;
    public static final Integer SYSTEM_TIMEOUT_ERR = 50002;
    public static final Integer BUSINESS_ERR = 60001;
    public static final Integer UNKNOWN_ERR = 99999;
}
```

3.3 在`BookServiceImpl`中触发自定义异常

```
package com.zsh.service.impl;

import com.zsh.controller.Code;
import com.zsh.dao.BookDao;
import com.zsh.domain.Book;
```

```java
import com.zsh.exception.BuisnessException;
import com.zsh.exception.SystemException;
import com.zsh.service.BookService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class BookServiceImpl implements BookService {

    @Autowired
    private BookDao bookDao;

    @Override
    public boolean save(Book book) {
        bookDao.save(book);
        return true;
    }

    @Override
    public boolean delete(Integer id) {
        bookDao.delete(id);
        return true;
    }

    @Override
    public boolean update(Book book) {
        bookDao.update(book);
        return true;
    }

    @Override
    public Book getById(Integer id) {
        if (id < 1) {
            throw new BuisnessException(Code.BUSINESS_ERR, "您输入的id不合法，请重新输入！");
        }

        try {
            int i = 1 / 0;// 模拟异常
        } catch (Exception e) {
            throw new SystemException(Code.SYSTEM_TIMEOUT_ERR, "服务器访问超时，请重试！", e);
        }
```

```
        return bookDao.getById(id);
    }

    @Override
    public List<Book> getAll() {
        return bookDao.getAll();
    }
}
```

3.4 重构 `ProjectExceptionAdvice`

```
package com.zsh.controller;

import com.zsh.exception.BuisnessException;
import com.zsh.exception.SystemException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

/**
 * 项目异常统一处理
 */
@RestControllerAdvice// REST风格
public class ProjectExceptionAdvice {

    @ExceptionHandler(SystemException.class)// 拦截系统异常并处理
    public Result handleSystemException(SystemException e) {
        // 1.记录日志
        // 2.发送消息给运维人员
        // 3.发送邮件给开发人员
        return new Result(e.getCode(), null, e.getMessage());
    }

    @ExceptionHandler(BuisnessException.class)// 拦截业务异常并处理
    public Result handleBusinessException(BuisnessException e) {
        return new Result(e.getCode(), null, e.getMessage());
    }

    @ExceptionHandler(Exception.class)// 拦截其他异常并处理
    public Result handleException(Exception e) {
        // 1.记录日志
        // 2.发送消息给运维人员
```

```
31          // 3.发送邮件给开发人员
32          return new Result(Code.UNKNOWN_ERR, null, "系统繁忙，请稍后再
    试！");
33      }
34  }
```

PartIV 前后端协议联调

一、更新`config`包下的核心配置类

1.新增`SpringMvcSupport`

```
1  package com.zsh.config;
2
3  import org.springframework.context.annotation.Configuration;
4  import
    org.springframework.web.servlet.config.annotation.ResourceHandlerRegist
    ry;
5  import
    org.springframework.web.servlet.config.annotation.WebMvcConfigurationSu
    pport;
6
7  @Configuration
8  public class SpringMvcSupport extends WebMvcConfigurationSupport {
9
10     @Override
11     // 放行静态资源
12     protected void addResourceHandlers(ResourceHandlerRegistry
    registry) {
13
     registry.addResourceHandler("/pages/**").addResourceLocations("/pages/
    ");
14
     registry.addResourceHandler("/css/**").addResourceLocations("/css/");
15
     registry.addResourceHandler("/js/**").addResourceLocations("/js/");
```

```
16
     registry.addResourceHandler("/plugins/**").addResourceLocations("/plug
    ins/");
17      }
18  }
```

2.重构`SpringMvcConfig`

```
1  package com.zsh.config;
2
3  import org.springframework.context.annotation.ComponentScan;
4  import org.springframework.context.annotation.Configuration;
5  import org.springframework.web.servlet.config.annotation.EnableWebMvc;
6
7  @Configuration
8  @ComponentScan({"com.zsh.controller", "com.zsh.config"})
9  @EnableWebMvc
10 public class SpringMvcConfig {
11 }
```

二、完善功能模块